
AI I Dakar

Guide de Déploiement

Application Web et Mobile (PWA)

Version	1.0 – Phase Pilote
Framework	Next.js 16 + TypeScript
Base de données	Prisma + SQLite
Plateformes	Web, Android (PWA), iOS (PWA)
Routes pilotes	Thiès / Thiènaba / Dakar

Mars 2026

Table des Matières

1. Vue d'ensemble du projet	4
1.1 Architecture technique	4
2. Déploiement sur Vercel	4
2.1 Préparation du dépôt GitHub	5
2.2 Configuration Vercel	5
2.3 Variables d'environnement	5
3. Migration de la base de données	6
3.1 Options recommandées	6
3.2 Procédure de migration vers Turso	6
4. Installation sur mobile (PWA)	7
4.1 Installation sur Android	7
4.2 Installation sur iOS	7
4.3 Fonctionnalités PWA incluses	7
5. Déploiement Docker (Alternative)	8
5.1 Commandes de déploiement	8
6. Configuration du domaine personnalisé	9
6.1 Configuration DNS	9
7. Checklist de déploiement	9

1. Vue d'ensemble du projet

Allô Dakar est une application de covoiturage conçue spécifiquement pour le marché sénégalais. Elle fonctionne à la fois comme une application web classique accessible via un navigateur, et comme une Progressive Web App (PWA) installable sur les téléphones Android et iOS. Cette double approche permet de couvrir l'ensemble des utilisateurs sans nécessiter une publication sur les stores d'applications, ce qui est particulièrement adapté pour la phase pilote qui se concentre sur les axes Thiès / Thiènaba et Dakar.

1.1 Architecture technique

L'application repose sur une architecture moderne et éprouvée, parfaitement adaptée au déploiement en production. Le framework Next.js 16 avec le mode App Router offre des performances optimales grâce au rendu côté serveur (SSR) et à la compilation Ahead-of-Time (AOT). La gestion des données est assurée par Prisma ORM qui fournit une abstraction propre et type-safe sur la base de données SQLite. L'interface utilisateur est construite avec le système de design shadcn/ui, garantissant une expérience cohérente et professionnelle sur tous les écrans.

Couche	Technologie	Rôle
Frontend	Next.js 16 + React 19	Interface utilisateur et navigation
Styling	Tailwind CSS 4 + shadcn/ui	Design responsive mobile-first
State	Zustand	Gestion d'état client-side
Backend API	Next.js API Routes	Authentification, CRUD trajets
Base de données	Prisma + SQLite	Stockage des données
PWA	Service Worker + Manifest	Installation et cache hors-ligne
Déploiement	Vercel / Docker	Hébergement production

Tableau 1 : Stack technique de Allô Dakar

2. Déploiement sur Vercel

Vercel est la plateforme de déploiement recommandée pour Allô Dakar. Elle est conçue nativement pour les applications Next.js et offre un déploiement en un clic avec une intégration GitHub transparente. Le plan gratuit est suffisant pour la phase pilote et inclut un certificat SSL automatique, un CDN mondial pour des temps de chargement rapides, et un domaine personnalisé en .vercel.app. Chaque push sur la branche main déclenche automatiquement un nouveau déploiement, ce qui simplifie considérablement le cycle de développement.

2.1 Préparation du dépôt GitHub

Avant de déployer sur Vercel, vous devez publier le code source sur GitHub. Si ce n'est pas déjà fait, initialisez un dépôt Git, ajoutez tous les fichiers, et poussez-les vers un nouveau dépôt GitHub. Assurez-vous que le fichier .gitignore est correctement configuré pour exclure node_modules, .next, et les fichiers de base de données locales qui ne doivent pas être versionnés.

```
git init
git add .
git commit -m "Initial commit - Allô Dakar"
git remote add origin https://github.com/VOTRE-USER/allo-dakar.git
git push -u origin main
```

2.2 Configuration Vercel

Connectez-vous sur vercel.com avec votre compte GitHub. Cliquez sur "New Project", sélectionnez votre dépôt Allô Dakar, et Vercel détectera automatiquement qu'il s'agit d'une application Next.js. Vous n'aurez qu'à configurer les variables d'environnement nécessaires avant de lancer le déploiement. Le processus complet prend moins de 5 minutes pour un déploiement initial.

2.3 Variables d'environnement

Les variables d'environnement sont essentielles pour le bon fonctionnement de l'application en production. Elles doivent être configurées dans le tableau de bord Vercel, dans la section Settings / Environment Variables. Voici les variables requises pour chaque étape du projet.

Variable	Valeur Développement	Valeur Production
DATABASE_URL	file:./db/custom.db	URL de votre BDD cloud

NEXTAUTH_SECRET	(optionnel)	Clé secrète 32+ caractères
NEXTAUTH_URL	http://localhost:3000	https://allo-dakar.vercel.app
NODE_ENV	development	production (auto)

Tableau 2 : Variables d'environnement requises

3. Migration de la base de données

La base de données SQLite utilisée en développement n'est pas adaptée à un environnement de production. En effet, SQLite stocke les données dans un fichier local, ce qui ne fonctionne pas avec l'architecture serverless de Vercel où chaque requête peut être traitée par un serveur différent. Vous devez migrer vers une base de données cloud qui offre une persistance des données et une meilleure scalabilité.

3.1 Options recommandées

Service	Type	Avantages	Tarif
Turso	SQLite Cloud	Migration facile depuis SQLite	Gratuit (500 MB)
Supabase	PostgreSQL	Auth, Storage, Real-time inclus	Gratuit (500 MB)
PlanetScale	MySQL	Branching de BDD, scalable	Gratuit (1 GB)
Neon	PostgreSQL	Serverless natif, auto-scale	Gratuit (0.5 GB)

Tableau 3 : Comparaison des services de base de données cloud

3.2 Procédure de migration vers Turso

Turso est l'option recommandée car elle utilise SQLite en backend, ce qui minimise les changements de code. La procédure se déroule en quelques étapes simples. D'abord, créez un compte sur turso.tech et installez la CLI Turso. Créez ensuite une base de données avec la commande `turso db create allo-dakar`. Obtenez l'URL de connexion avec `turso db show allo-dakar --url` et le token d'authentification avec `turso db tokens create allo-dakar`. Enfin, mettez à jour votre fichier `.env` avec ces nouvelles valeurs.

```
# Dans prisma/schema.prisma, remplacer le provider :
datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}
```

```
}  
  
# Puis exécuter :  
bun run db:push  
bun run db:generate
```

4. Installation sur mobile (PWA)

La Progressive Web App (PWA) permet aux utilisateurs d'installer All□ Dakar directement sur leur téléphone, comme une application native, sans passer par les stores d'applications. L'application dispose d'un Service Worker pour le cache hors-ligne, d'un manifest.json pour les métadonnées, et d'un prompt d'installation automatique. L'expérience utilisateur est quasiment identique à une application native, avec un lancement depuis l'écran d'accueil, un écran de chargement personnalisé, et un fonctionnement en mode plein écran.

4.1 Installation sur Android

Sur Android, l'installation est entièrement automatique. Lorsqu'un utilisateur visite le site pour la première fois avec Google Chrome, une bannière d'installation apparaît automatiquement après quelques secondes. L'utilisateur n'a qu'à cliquer sur le bouton "Installer" pour ajouter All□ Dakar sur son écran d'accueil. Une ic□ne personnalisée est créée et l'application s'ouvre en mode plein écran, sans la barre d'adresse du navigateur, offrant une expérience immersive.

4.2 Installation sur iOS

Sur iOS, le processus est légèrement différent car Apple ne supporte pas encore le prompt d'installation automatique. L'utilisateur doit manuellement ajouter l'application à son écran d'accueil. Pour ce faire, il doit ouvrir le site dans Safari, appuyer sur l'ic□ne de partage (le carré avec une flèche vers le haut), puis sélectionner "Sur l'écran d'accueil". Une fois ajoutée, l'application se comporte comme une app native avec un lancement rapide et un fonctionnement hors-ligne partiel.

4.3 Fonctionnalités PWA incluses

Fonctionnalité	Description	Statut
----------------	-------------	--------

Service Worker	Cache des ressources et mode hors-ligne	Actif
Manifest.json	Métadonnées de l'app (nom, icônes, couleurs)	Configuré
Prompt d'installation	Bannière automatique (Android) / iOS guide	Actif
Icônes PWA	8 tailles de 72x72 à 512x512	Générées
Thème couleur	Barre de statut verte (#006233)	Configuré
Mode standalone	Plein écran sans barre du navigateur	Configuré
Background Sync	Synchronisation des réservations	Prêt

Tableau 4 : Fonctionnalités PWA implémentées

5. Déploiement Docker (Alternative)

Si vous préférez un contrôle total sur l'infrastructure, le déploiement via Docker est une excellente alternative. Cette approche est recommandée si vous disposez d'un serveur VPS (Virtual Private Server) chez un fournisseur comme OVH, DigitalOcean ou AWS EC2. Le fichier Dockerfile est déjà configuré dans le projet pour créer une image optimisée avec un build multi-étapes qui minimise la taille finale de l'image tout en incluant toutes les dépendances nécessaires.

5.1 Commandes de déploiement

Le processus de déploiement Docker se résume à quelques commandes simples. D'abord, construisez l'image avec `docker build -t allo-dakar .` puis lancez le conteneur avec la commande appropriée en exposant le port 3000. Pour la production, il est recommandé d'utiliser un reverse proxy comme Nginx ou Caddy pour gérer le SSL et le routage HTTP.

```
# Construction de l'image
docker build -t allo-dakar .

# Lancement du conteneur
docker run -d -p 3000:3000 --name allo-dakar \
  -e DATABASE_URL="file:./db/custom.db" \
  allo-dakar

# Avec Docker Compose (recommandé)
docker-compose up -d
```


6. Configuration du domaine personnalisé

Pour une image professionnelle, il est recommandé d'utiliser un domaine personnalisé comme `allodakar.sn` au lieu du sous-domaine Vercel par défaut. Au Sénégal, vous pouvez enregistrer un domaine `.sn` auprès du NIC Sénégal ou utiliser un registrar international comme Namecheap ou OVH pour un domaine `.com` ou `.sn`. La configuration DNS consiste à ajouter un enregistrement CNAME pointant vers `cname.vercel-dns.com` dans votre zone DNS. Vercel gère automatiquement le certificat SSL via Let's Encrypt.

6.1 Configuration DNS

Type	Nom	Valeur
CNAME	@ ou www	cname.vercel-dns.com
A	@	76.76.21.21
TXT	_vercel	vc-domain-verify=allodakar

Tableau 5 : Configuration DNS pour Vercel

7. Checklist de déploiement

Voici une checklist complète à suivre pour garantir un déploiement réussi de Allodakar en production. Chaque étape doit être validée avant de passer à la suivante. Cette approche méthodique permet de minimiser les risques et d'assurer une mise en production fiable et professionnelle.

Étape	Action	Vérification
1	Pousser le code sur GitHub	Le dépôt est public/privé sur GitHub
2	Créer le projet Vercel	Le dashboard Vercel est configuré
3	Configurer la BDD cloud	Turso/Supabase est connecté
4	Mettre les variables d'env	DATABASE_URL est correct
5	Lancer le déploiement	Le build réussit sans erreur
6	Tester l'application	Login, recherche, réservation OK
7	Tester la PWA mobile	Installation sur Android/iOS OK

8	Configurer le domaine	Le domaine est accessible
9	Vérifier le SSL	HTTPS actif, certificat valide
10	Effectuer les seed data	Données de test présentes

Tableau 6 : Checklist de déploiement production

8. ☐volutions futures recommandées

Une fois la phase pilote validée et l'application stable en production, plusieurs évolutions sont envisageables pour améliorer l'expérience utilisateur et étendre le service. Voici les principales pistes de développement à considérer pour les futures versions de l'application.

☐ volution	Description	Priorité
App native (Capacitor)	Envelopper la PWA dans une app native pour les stores	Haute
Paieement mobile	Intégration Orange Money / Wave pour le paiement en ligne	Haute
Notifications push	Alertes pour les nouveaux trajets et confirmations	Haute
Carte interactive réelle	Intégration Google Maps pour la vue trajet	Moyenne
Système de notation	Avis passager/chauffeur après chaque trajet	Moyenne
Extension régions	Ajouter Saint-Louis, Kaolack, Ziguinchor	Moyenne
Géolocalisation	Suivi en temps réel du véhicule	Basse

Tableau 7 : ☐volutions futures recommandées